

김명성

Frontend Engineer · 경력 5년 7개월

레거시를 무중단으로 전환하고, 판단의 근거를 RFC로 남겨 팀의 표준으로 정착시키는 프론트엔드 엔지니어입니다.

010 8663 2663 maeng2418@gmail.com github.com/maeng2418 maeng2418.github.io

핵심 성과

레거시 전환 4년 완주

Vue/Nuxt → React 점진 전환을 RFC 합의로 주도. 레거시 코드 비중 2022년 100% → 2026년 1.1%까지 무중단 완주

빌드 589.24s → 캐시 적중 1.10s

Turborepo 도입과 빌드 설정 정리. 원시 측정 로그를 사내 문서에 기록

초기 로드 3.4MB → 1.5MB (55.9% 감소)

DOMContentLoaded 346ms → 121ms. AI가 제시한 결론을 계속으로 반증하고 원인을 재정의한 결과

테스트 부재 모노레포에 E2E 체계 도입

Playwright 기반 회귀 검증을 0에서 구축해 CI에 편입

컨벤션을 문서가 아닌 도구로 강제

ADR을 AI 스킬로, 반복 지적을 린트 규칙으로, 검증을 CI 게이트로 승격

경력사항



놀유니버스(야놀자)

2022.09.26 – 재직중 (3년 10개월)

숙박 파트너가 요금·재고를 직접 관리하는 글로벌 B2B 콘솔 · PC/Mobile/Admin



IGAWorks
(Adpopcorn)

2021.07 – 2022.09

애드테크 데이터 플랫폼 · SSP 콘솔 리뉴얼(NextJS→React), 광고 성과 대시보드, Svelte 랜딩



집싸 (파트너 앱)

2020.12 – 2021.07

이사 O2O 하이브리드 앱 (Cordova) · JS→TS 전환, Atomic Design 시스템, 컴포넌트 CLI

보유 스킬

주력 — TypeScript · React · Next.js

상태·폼 TanStack Query · React Hook Form · Zod

빌드·테스트 Vite · Turborepo · pnpm · Playwright · Vitest · Storybook

스타일·구조 MUI · Tailwind CSS · Emotion · FSD

인프라 GitHub Actions · Docker/EKS

주요 프로젝트

Vue/Nuxt → React 점진 전환 — 전환 아키텍처 설계 및 리드

놀유니버스 · 숙박 파트너 B2B 콘솔 · 레거시 7레포 543건 + 신규 10레포 병존 · FE 3명 협업

문제

레거시 코드베이스와 신규 React 앱이 병존하는 상태에서 서비스 중단 없이 전환해야 했습니다. 2022년 입사 당시 제가 스스로 약점으로 지목했던 것이 바로 이 Vue/Nuxt 스택이었습니다.

판단 — 전환 아키텍처

전환 전략을 RFC로 먼저 정리해 팀 합의를 거쳤습니다. 한 번에 옮기지 않고 Phase 0에서 인프라·인증·라우팅 공통 기반을 선행한 뒤 도메인을 단계별로 전환하는 순서를 택했습니다.

실행

nginx 경로 라우팅과 Istio VirtualService 경로 분기로 구·신 앱 공존 구간을 확보했습니다. 대형 전환은 Part1~4로 분할했고, 신규 구현과 레거시 제거를 짝으로 실행했습니다. 마지막 Nuxt 앱 철거 단계까지 사전 설계에 포함했습니다.

결과

레거시 PR 비중 2022년 100% → 2023년 69.4% → 2024년 22.3% → 2025년 6.5% → 2026년 1.1%. 약점으로 적었던 스택을 4년에 걸쳐 완주했습니다.

성능 개선 — AI 초안을 검증하고 기각

모바일 웹 · 번들링 전략 재정의

문제

특정 페이지에서 3.44MB 거대 청크가 로드되고 있었습니다. LLM에 분석을 맡겼더니 React.lazy 코드 스플리팅으로 대폭 감소했다는 결론을 제시했습니다.

판단

초안이 스스로 적어 둔 측정 한계를 발견했습니다 — 개선 전후의 DOMContentLoaded와 Load 값이 동일하게 측정됐고, 초안 자신이 "캐시가 적용된 상태일 가능성이 높다"고 인정하고 있었습니다. 코드 스플리팅 유지와 단일 번들 두 안을 놓고 다시 비교했습니다.

실행

AI의 결론을 기각하고 문제를 재정의했습니다. 쟁점은 스플리팅 대 모노리식이 아니라 과도한 용량을 유발한 근본 원인이었습니다.

거대 청크를 제거한 뒤 직접 재측정했고, 모든 페이지가 1.71MB를 로드하는 트레이드오프를 인정하되 이전의 페이지별 편차보다 예측 가능하다는 근거로 단일 번들을 택했습니다.

결과

초기 로드 3.4MB → 1.5MB(55.9% 감소), DOMContentLoaded 346ms → 121ms, Load 487ms → 349ms. 근거 범위는 모바일 앱 1개의 클라이언트 측정입니다.

클라이언트 로그 계측 DevTools 구축 — 확인 비용을 코드에서 화면 위로

놀유니버스 · 파트너 콘솔 로그 정의 304건 · domestic(MUI)·outbound(NDSP) 4개 앱 배선 · 신규 의존성 0

문제

로그 정의 304건이 앱별 생성 코드 1만 줄에 흩어져, 무엇이 심겼고 실제 쌓이는지 코드 밖에서 확인할 방법이 없었습니다. 죽은 정의, 센더와 정의의 drift(호출 파일 174개), StrictMode 이중 발화가 누적됐습니다.

판단

확인 비용을 화면 위로 옮기기로 했습니다. 인스펙터는 앱 디자인시스템에 의존하지 않는 vanilla DOM으로 구현해 MUI·NDSP 양쪽 앱군에 동일하게 넣고, 수집 동작 변경은 파일럿 1화면으로 한정해 적재 데이터 영향을 차단했습니다.

실행

생성 센더 코드를 역파싱해 카탈로그 5종(사람용 md·상세 json·적재 대조 SQL·런타임 메타·ref 타입)을 결정적 산출물로 자동 생성했습니다. 실시간 발화 피드·30일 실수집량 오버레이·txtpb 생성 모드를 갖춘 dev 전용 인스펙터와, 태깅 한 줄이 곧 수집이 되는 자동 발화 런타임을 구축했습니다.

결과

자동 발화가 기존 센더와 페이로드 바이트 동일함을 실측 대조했고, 프로덕션 번들 추가분은 슬림 메타 96KB뿐입니다. Databricks PAT는 로컬 dev 프록시로 격리해 신규 의존성 0으로 구현했으며, 채택·마이그레이션 논의는 컨벤션 RFC로 분리했습니다.

주요 프로젝트 (계속)

AI 개발 인프라 팀 표준화

사내 Claude 플러그인 마켓플레이스 신설 · 소비 레포 5개

문제

개발 도구가 모노레포에 종속돼 있었고, 개인 자산과 팀 자산의 경계가 불분명했습니다. 자율 설치에 맡기면 필수 규칙이 누락될 위험도 있었습니다.

판단

대안 9개를 대조했습니다 — 레포 종속축 4개와 공용축 5개. 각 안의 장단점과 제안 사유를 문서에 별기했습니다. 팀 공용 도구는 잘못된 변경 하나가 전체 작업을 막을 수 있어, CI·권한·컨벤션 안전망을 먼저 깬 뒤 기능을 추가하는 순서를 택했습니다.

실행

.claude/project-plugin과 gitignore 화이트리스트로 레포 종속 자산을, 별도 레포 마켓플레이스로 공용 자산을 분리하는 2축 구조를 채택했습니다.

결과

사내 마켓플레이스를 신설해 플러그인 8종을 배포했고, 소비 레포 5개에 적용하며 온보딩 문서에 편입했습니다.

지식 시스템 재설계 — LLM-Wiki 전환

대형 모노레포 지식 빌드 파이프라인

문제

도메인당 스킴 1개 구조여서 같은 도메인을 10세션 작업하면 draft가 10개로 파편화됐습니다.

판단

대안 3종을 기각 근거와 함께 남겼습니다 — 벡터DB/임베딩은 마크다운과 grep으로 충분해서, 혹 자동주입은 오탐과 복잡도 때문에, draft PR 전환은 리뷰어 자동지정을 유지하기 위해 각각 채택하지 않았습니다.

실행

ADD/UPDATE/NOOP 3분기 판정 구조와 자기증식 방지 2계층 가드를 설계했습니다.

결과

독립 감사 점수 0.52 FAIL에서 0.87 PASS로, evaluator 93/85/90/92, 테스트 129건 green.

해외 파트너센트럴 — 판매 캘린더 아키텍처 설계 · 데이터 변환 표준

해외 숙박 파트너 대상 PC·Admin · 다국어/다통화 · FE 2명 + PM·BE 2명·PPD·QA 협업

문제

해외 숙소 파트너가 요금제·재고·판매 상태를 관리하는 화면군을 신규 구축해야 했습니다. 통화 소수점 자리와 요금제 다국어 표기 같은 공통 정책이 화면마다 제각각 구현되면, 정책이 바뀔 때마다 모든 화면을 찾아 고쳐야 하는 구조가 됩니다.

판단 — 아키텍처

공통 정책(통화 소수점, 요금제 다국어)을 화면이 아니라 하위 레이어로 내리고 화면은 조합만 하도록 FSD 레이어를 분리했습니다. 기준은 "정책 변경의 영향 범위를 좁게 유지"였습니다.

폼·요청 변환에도 같은 원칙을 적용해 역할(검증 혹 / 요청 변환 순수함수)·위치(feature 하위 co-locate)·형태 3축 규칙을 정했고, 억지 추상화를 막기 위해 passthrough 요청은 래퍼를 만들지 않는다는 경계까지 명문화했습니다.

실행

전담 디자이너가 없는 구간에서는 시안을 여러 개 만들어 동료들과 공유하고 피드백을 받아 안을 채택했습니다.

컨벤션 전면 적용은 Stacked PR 6단계로 나눠 각 단계마다 타입체크와 린트를 통과시켰습니다. 요청 객체는 쿼리 캐시 키의 구성요소이므로 shape과 값 을 바이트 단위로 보존해 캐시 동작이 바뀌지 않도록 했습니다.

결과

캘린더·기본가격·Daily Rate List가 동일한 인원요금·Cap 값을 일관되게 표시하도록 정렬했고, 신규 기능은 피쳐플래그로 단계 적용해 기존 운영을 깨지 않으면서 노출했습니다. 정찰 시점 컨벤션 준수율 75~80%를 전수 정규화했습니다.

이전 경력 상세



IGAWorks (Adpopcorn)

2021.07 - 2022.09 · 애드테크 데이터 플랫폼 · Frontend Engineer

담당

모바일 광고 SSP 플랫폼(Adpopcorn) 콘솔 전면 리뉴얼, 뉴스 광고 플랫폼(콘픽) 개발, 광고 성과 분석 대시보드 구축.

판단·실행

기존 NextJS SSR 구조가 콘솔 성격에 맞지 않아 SPA로 전환하며 불필요한 서버 렌더링 오버헤드를 제거했습니다. React-Query와 Recoil로 상태관리를 재설계해 중복 API 호출을 정리하고 번들을 최적화했습니다.

랜딩 페이지는 컴파일 타임에 프레임워크가 사라지는 Svelte를 선택해 로드 성능을 확보했습니다.

결과

Recharts + SVG 기반으로 캠페인별 매출·클릭·노출을 실시간 시각화하는 대시보드를 제공했고, AWS CodePipeline CI/CD와 ECS/EKS 컨테이너 배포 환경을 구성해 무중단 배포 파이프라인을 운영했습니다.



짐싸 (파트너 앱)

2020.12 - 2021.07 · 이사 O2O 플랫폼 · Frontend Developer

담당

이사 매칭 하이브리드 앱(짐싸 파트너)을 Cordova 기반 iOS/Android 크로스 플랫폼으로 개발. 디자인 2.0 리뉴얼 담당.

판단·실행

JavaScript로 작성된 코드베이스를 TypeScript로 전환하고 Lazy-Loading을 적용해 초기 로딩을 개선했습니다. UI 일관성 확보를 위해 Atomic Design 컴포넌트 시스템을 구축하고 Storybook으로 검증 체계를 붙였으며, 다크모드까지 대응했습니다.

결과

ESLint·Prettier·Husky로 코드 품질 도구를 정착시키고, Atomic Design 보일러플레이트를 자동 생성하는 컴포넌트 CLI를 개발해 반복 작업을 줄였습니다.

표준 수립

컨벤션을 문서가 아닌 도구로 강제

읽히길 기대하는 문서는 지켜지지 않는다고 보고, 규칙을 실행 계층으로 내렸습니다. ADR을 AI 스킴으로 이관해 작업 시점에 자동 적용되게 하고, 반복 지적 사항은 린트 규칙으로, 빌드·타입 검증은 CI 게이트로 승격했습니다.

컨벤션 스킴화 — 타입 파생 규칙 · 빈 값 표기 · 테이블 셀 이동 · frontmatter 정책

각 컨벤션을 문서 대신 스킴으로 만들어 팀 전체에 일괄 적용했습니다. 규칙이 바뀌면 스킴 한 곳만 고치면 됩니다.

신규 레포 협업 규약 이식

CODEOWNERS · PR 템플릿 · GitHub Actions 워크플로 3종 · husky 혹 3종 · commitlint를 신규 레포 첫 PR로 세팅해, 첫 커밋부터 규약이 강제되도록 했습니다.

구조 개선 · 안전망 구축

정책을 코드에서 선언으로 — 구조 설계로 변경 지점 11개 파일 → 1곳

판매 잠금 기준이 11개 파일에 흩어져 있어, 정책 하나를 바꾸려면 모든 사용처를 찾아 개별 수정해야 했습니다. 누락되면 화면마다 다르게 동작하는 버그가 됩니다. 채널별 규칙을 코드 분기가 아닌 정책 선언으로 분리해 단일 지점으로 모았고, 이후 규칙 변경은 선언 한 곳 수정으로 끝나며 사용처는 손대지 않습니다.

locale 타입 3중 중복 제거 — 기존 호출부 20여 곳 무수정

다국어 텍스트 타입이 세 곳에 각각 정의돼 있어 한 곳만 고치면 나머지와 어긋났습니다. 모듈 확장 슬롯 패턴으로 단일 정의에 통합하되, 기존 소비처 20여 곳이 그대로 컴파일되도록 설계했습니다. 대규모 정리를 한 번에 끊지 않고 점진 적용 가능한 형태로 만든 사례입니다.

테스트 부재 상태에서 E2E 회귀 검증 체계 구축

테스트 코드가 전무한 모노레포였습니다. Playwright를 도입하며 도구만 붙이지 않고, 로그인 세션과 데이터 준비를 픽스처로 표준화해 테스트마다 사전 조건을 다시 만들지 않도록 했습니다(신규 2,833라인). 핵심 플로우 회귀 검증을 CI에 편입해 배포 전 자동 실행됩니다.

배포 번들에 노출되던 인증 토큰 제거

클라이언트 번들 산출물에 Sentry auth token이 포함되어 배포되고 있었습니다. 빌드 시점에만 필요한 값이 런타임 산출물로 새어 나간 경우로, 주입 경로를 분리해 번들에서 제거했습니다.

활동 · 문서화

사내 기술 문서 작성

Playwright E2E 도입, Turborepo 적용, 차트 라이브러리 교체, 공통 컴포넌트 통합, 요금제 도메인 명세 등을 Confluence에 작성. 릴리즈 Launch Plan 4회, 파트너센터 제품·운영 가이드 60건 이상.

RFC · Tech Spec 작성

제휴입점 관리자 React 전환 전략(대안 11개 대조), NDSP 컴포넌트 API 일관성, 인프라 경로 분기 설계.

우아한 테크 캠프 3기 수료 · 2020.07 – 2020.08

우아한형제들 주관 8주 웹 개발 교육. 코드 리뷰와 페어 프로그래밍 수행.

학력



서울과학기술대학교 컴퓨터공학과 학사 졸업

2015.03 – 2021.02